

Prompt

I don't want to change the quality of the shader render too much. What I'm looking for is innovative shortcuts in the shader math. This does not have to be a mathematically perfect fractal render. It's not a scientific visualisation but one that is intended for live VJ'ing at a high frame rate.

Yes — for a VJ-oriented renderer, I'd focus on perceptual shortcuts inside the distance estimator and raymarch, not conventional "quality settings."

The key observation is that a 40-iteration Mandelbox contains detail far smaller than a screen pixel for most of the image. Computing it is mathematically faithful but visually wasted. The shader can be made much faster by stopping or simplifying work once the remaining detail cannot affect the current pixel.

The strongest candidates are:

1. Pixel-footprint iteration cutoff

Instead of always running the requested iteration count, calculate how large one screen pixel is at the ray's current distance. Mandelbox details shrink roughly exponentially per iteration.

Once the next level of detail is smaller than the projected pixel footprint, stop the fractal iteration loop for that ray.

Conceptually:

```
pixelWorldSize ≈ rayDistance * tan(fieldOfView / screenHeight)
nextDetailSize ≈ baseSize / pow(abs(Scale), iteration)

if (nextDetailSize < pixelWorldSize)
    break;
```

Close-up areas still get high detail. Distant faces and peripheral geometry automatically use fewer iterations, because additional detail is invisible anyway. This is a particularly good fit for VJ visuals: it maintains the apparent complexity without spending work on sub-pixel structure.

2. Mandelbox escape bailout

The current `DE()` loop always executes all requested iterations. A Mandelbox orbit can often be treated as escaped once `dot(z, z)` exceeds a chosen bailout radius squared.

At that point, the remaining folds contribute little to the visible surface estimate, so use the current `length(z) / dr` estimate and exit.

```
if (dot(z, z) > bailoutSquared)
    break;
```

This is an approximation, but for visual use it will usually be hard to spot and can save many iterations in empty space or rays that travel away from the interesting structure.

3. Screen-space adaptive hit threshold

The shader uses a fixed `epsilon = 0.0001`. That forces extremely fine raymarch convergence even when the object is far away and a pixel covers a much larger world-space area.

Use a distance-aware epsilon instead:

```
float adaptiveEpsilon = max(0.0005, rayTravelDistance * pixelAngularSize);
if (distanceEstimate < adaptiveEpsilon)
    break;
```

This can remove a substantial number of final tiny raymarch steps while retaining a crisp image at the resolution being displayed. It is one of the most visually forgiving optimizations.

4. Reduce iterations for rays that are already “uninteresting”

The shader currently calculates the full orbit trap for every DE evaluation. But for pixels that land in broad, low-contrast regions, a high-fidelity orbit trap is not visibly important.

A stylised shortcut:

- Run full iterations near silhouettes/high-contrast areas.
- Use a lower iteration cap for broad interior surfaces.
- Optionally introduce a very subtle animated colour/noise treatment to mask the simplification—which can actually improve live VJ aesthetics.

A cheap first pass can classify the ray based on approximate distance or early orbit values. Only “interesting” pixels receive the expensive refinement pass.

5. Orbit-trap decimation

This is a small but very safe mathematical shortcut. The current hot loop computes two square roots per iteration:

```
if (length(z) < length(trap))
    trap = z;
```

It should compare squared lengths:

```
if (dot(z, z) < dot(trap, trap))
    trap = z;
```

That avoids square roots in the tightest loop. You could also update the orbit trap only every second iteration at high depths; this slightly changes colour detail but not the geometry.

6. Iteration “tiering” rather than a hard quality drop

Rather than treating 40 iterations as 40 equally valuable passes:

- Iterations 1–8: geometry and major silhouette
- 9–16: recognisable internal repetition
- 17–24: fine decoration
- 25+: mostly micro-detail / colour variation at normal viewing distances

A VJ profile could always run the first 12–16, then selectively permit later tiers only where they survive the pixel-footprint test. This gives the look of a high-iteration fractal without paying the high-iteration cost across the whole frame.

7. Sparse expensive-pixel sampling with a VJ-friendly dither

Rather than rendering lower resolution globally, render expensive refinement only for a rotating/interleaved subset of pixels:

- Every pixel gets a fast low/mid-iteration base estimate.
- A blue-noise or checkerboard subset gets high-iteration refinement each frame.
- The pattern rotates over time.

For static imagery this would be undesirable, but for moving VJ content it can read as texture or shimmer rather than a defect. It preserves apparent full detail and can cut high-depth work dramatically.

8. Deliberately stylised distance-estimator approximation

The current DE computes the full orbit each raymarch step. For a live visual tool, you could use:

- Full DE only every second or third raymarch step.
- A conservative extrapolation of the prior DE for intermediate steps.
- Full DE again whenever the ray gets near a surface.

That is less mathematically pure, but the error tends to manifest as organic surface variation—often visually desirable in a VJ context.

My recommended experimental order would be:

1. Pixel-footprint iteration cutoff
2. Adaptive hit epsilon
3. Escape bailout
4. Squared-length orbit-trap comparison
5. Optional sparse high-detail refinement / dither mode

The first three directly reduce shader maths while retaining most perceived image quality. They also compose well: the renderer naturally spends its compute budget where the audience can actually see the fractal complexity.